

# 比赛方法梳理与后续优化方向

Adam老师

2026/05/24





# 目录

---

- 比赛 最优/主流 解法梳理
- 后续优化方向



# 最优/主流 解法

- 目标：把推理题先程序化，再让 Nemotron 稳定复现轨迹
- SFT to maximize minimum logprob: 不是训练模型“多想几条路”，而是让正确 trace 上最不稳定的 token 也变得足够高概率
- 程序负责找 policy, SFT 负责让模型在固定格式里稳定执行 policy





# 不是 RL 或蒸馏，而是 Nemotron 可以被训练成“简单计算机”

对可解问题已经知道近似最优策略，模型只需要在正确轨迹上逐 token 跟随

## 程序化题解 / CoT

人和代码先把规则拆清：Roman numeral 枚举、单位换算分步乘除、cipher 字符映射、bit 按位归纳。

## SFT 学习轨迹

训练目标是让 trace 上每个关键 token 可复现；LoRA 不需要发明策略，只要学会执行。

## temperature=0 输出

评测采样温度为 0，最终看最可能 token 是否连续正确，因此 min logprob 是更直接的风险信号。

### ● 明确押注

- Nemotron LoRA 后可以像简单计算机一样执行固定推理程序。
- 设计更好的 chain-of-thought，尤其是 bit manipulation。

### ● 明确不押注

- 不押注 reinforcement learning：solvable problems 已有正确 policy。
- 不押注 distilling larger models：最优 policy 应该由程序直接生成。
- 不追求 CoT 多样性：只要答案正确，所有 trace 长得一样也可以。



# CoT 不是越像人类越好，而是越确定、越容易被模型逐 token 学会越好

把 CoT 当成可调试的程序输出：每个 token 尽量只依赖少量来源，且能通过日志定位低概率点。

## Deterministic

评测 temperature=0；只需让正确 token 成为最高概率 token。

## Simple

把除法拆成加减；把匹配拆成逐项分类，降低单步计算负担。

## Coverage

稀有操作也要训练；例如 Wonderland 词表不在首个匹配处提前停止。

## Length limit

完整 CoT 必须在比赛限制内；明确控制在 7680 tokens 以下。

## Tokenization-aware

避免让模型处理弱项：字符、符号、字母可能和其他 token 合并。

## Generalizable

除 Cipher 词表等必要信息外，不把方案设计成记忆训练集答案。

## 训练目标的翻译

平均 loss 下降不够；真正危险的是 trace 里最低概率的那个 token。一旦它在相似上下文中不是最高概率，temperature=0 推理就会偏离模板，后续 token 会连续错。

objective: maximize  $\min_t \log p(\text{token}_t | \text{prefix})$



# 七类题的策略差异很大，但都被压成“程序生成 trace，再让模型复现”

| 题型               | 任务本质                           | 作者策略                                                              | 目标 solve rate    |
|------------------|--------------------------------|-------------------------------------------------------------------|------------------|
| Numeral          | Roman numeral -> integer       | 训练集范围 1-100；直接枚举 numerals，同时保留 step-by-step trace。                | 100.0% 1576/1576 |
| Unit conversion  | 标量换算 + 乘法                      | 先找 scalar，再把乘除拆成多步，降低模型低级算术错误。                                    | 100.0% 1594/1594 |
| Gravity          | 物理量标量计算                        | 和单位换算类似：找 scalar，再分步乘除，减少一次性计算压力。                                 | 100.0% 1597/1597 |
| Cipher           | Wonderland substitution cipher | 从 examples 推字符映射，整理未映射字符，再完成 decrypt。                             | 100.0% 1576/1576 |
| Bit manipulation | 8-bit 输入输出规则归纳                 | 按 output bit 拆分布尔规则搜索；这是作者的主要差异化来源。                               | 85.1% 1364/1602  |
| Equation         | 二元数字运算规则                       | 枚举 4 种反转变换和 32 个 operator，先匹配高频 8 个，再匹配低频 24 个。                   | 76.6% 561/732    |
| Cryptarithm      | 符号/文字算术猜测                      | 没有完全解决；降级为识别 concatenation / reverse concatenation，缺失时按 concat 猜。 | 7.9% 65/823      |

easy tasks 的目标是接近 100%；真正决定上限的是 bit manipulation 与 cryptarithm 的新增洞察。



# 以 Equation Numeric (Deduce) 为例

- 拆成两个两位数、一个运算符、一个输出；
- 系统性搜索：
  - 操作数变换
  - 常见操作符 + 罕见操作符

In Alice's Wonderland, a secret set of  
 $45^{\wedge}08 = 0234$   
 $37\}95 = 131$   
 $99^{\wedge}88 = 2178$   
Now, determine the result for:  $52\}51$

ANSWER:

93

样例被拆成结构化记录

$45^{\wedge}08 = 0234$



$37\}95 = 131$



$99^{\wedge}88 = 2178$



路由判断

query = { a: 52, op: }, b: 51 }

1 先找同运算符样例:  $37\}95 = 131$

2 只用这个样例尝试归纳 } 的动作序列



# 目录

- 候选规则由三层组成：操作数是否反转、使用哪类运算、结果是否反转。
- 命中后，把这组动作记为 } 的可用程序。





# 最后执行 query 计算

前两页已经确定了 } 的动作序列；现在对新输入执行同一套程序。

```
program_} = reverse_operands -> add_minus_1 ->  
reverse_result
```

**1** 输入

**52}51**

保留 } 程序

**2** 反转操作数

**25, 15**

52 -> 25; 51 -> 15

**3** 加法减一

**25+15-1=39**

中间结果 39

**4** 反转结果

**93**

39 -> 93

所以系统最终输出

**boxed{93}**



# 示例

<https://nemotron.huikang.dev/synthetic.html?problem=07cbed38>

Looking at the question

52}51 -> }

The question operator is found in the examples.

Looking at operator **【}】** [37}95 = 131]:

Trying common operations reversed operands [37->73 95->59] and reversed result [expected (73,59)->131]:

concatenation  $f(73, 59) = 73 \parallel 59 = 7359$  -rev-> 9537 wrong

reverse concatenation  $f(73, 59) = 59 \parallel 73 = 5973$  -rev-> 3795 wrong

addition  $f(73, 59) = 73 + 59 = 132$  -rev-> 231 wrong

absolute difference  $f(73, 59) = |73 - 59| = 14$  -rev-> 41 wrong

negated absolute difference  $f(73, 59) = -|73 - 59| = -14$  -rev-> -41 wrong

subtraction (a-b)  $f(73, 59) = 73 - 59 = 14$  -rev-> 41 wrong

reverse subtraction (b-a)  $f(73, 59) = 59 - 73 = -14$  -rev-> -41 wrong

multiplication  $f(73, 59) = (70 + 3) * 59 = 70 * 59 + 3 * 59 = 4130 + 177 = 4307$  -rev-> 7034 wrong

Trying common operations on identity [37 95] [expected (37,95)->131]:

concatenation  $f(37, 95) = 37 \parallel 95 = 3795$  wrong

reverse concatenation  $f(37, 95) = 95 \parallel 37 = 9537$  wrong

addition  $f(37, 95) = 37 + 95 = 132$  wrong

absolute difference  $f(37, 95) = |37 - 95| = 58$  wrong

negated absolute difference  $f(37, 95) = -|37 - 95| = -58$  wrong

subtraction (a-b)  $f(37, 95) = 37 - 95 = -58$  wrong

reverse subtraction (b-a)  $f(37, 95) = 95 - 37 = 58$  wrong

multiplication  $f(37, 95) = (30 + 7) * 95 = 30 * 95 + 7 * 95 = 2850 + 665 = 3515$  wrong

Trying common operations reversed operands [37->73 95->59] [expected (73,59)->131]:

concatenation  $f(73, 59) = 73 \parallel 59 = 7359$  wrong

reverse concatenation  $f(73, 59) = 59 \parallel 73 = 5973$  wrong

addition  $f(73, 59) = 73 + 59 = 132$  wrong

absolute difference  $f(73, 59) = |73 - 59| = 14$  wrong

negated absolute difference  $f(73, 59) = -|73 - 59| = -14$  wrong

subtraction (a-b)  $f(73, 59) = 73 - 59 = 14$  wrong

reverse subtraction (b-a)  $f(73, 59) = 59 - 73 = -14$  wrong

multiplication  $f(73, 59) = (70 + 3) * 59 = 70 * 59 + 3 * 59 = 4130 + 177 = 4307$  wrong



# 后续优化方向

| Category                | Found | Total | Accuracy |
|-------------------------|-------|-------|----------|
| bit_manipulation        | 1364  | 1602  | 85.1%    |
| cipher                  | 1576  | 1576  | 100.0%   |
| cryptarithm_deduce      | 54    | 659   | 8.2%     |
| cryptarithm_guess       | 11    | 164   | 6.7%     |
| equation_numeric_deduce | 540   | 596   | 90.6%    |
| equation_numeric_guess  | 21    | 136   | 15.4%    |
| gravity                 | 1597  | 1597  | 100.0%   |
| numeral                 | 1576  | 1576  | 100.0%   |
| unit_conversion         | 1594  | 1594  | 100.0%   |
| TOTAL                   | 8333  | 9500  | 87.7%    |

## cryptarithm\_deduce

In Alice's Wonderland, a secret set of

/}-\` = ]

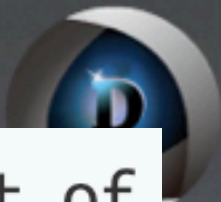
]( -: = -&/

\]+&( = &(\]

\#-{} = -#

:{\*#\ = ((`{

Now, determine the result for: :\+&/



深度之眼  
deepshare.net

## cryptarithm\_deduce

In Alice's Wonderland, a secret set of

&`\*'& = &&@`

&}\*}& = !{}

'?-{& = -<!

Now, determine the result for: '{+@`

ANSWER:

&{'

## cryptarithm\_guess

In Alice's Wonderland, a secret set of

06-41 = 47

45)45 = 0

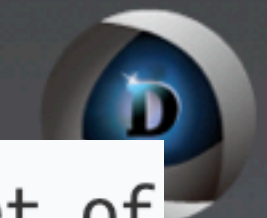
41)12 = 7

Now, determine the result for: 43<61

ANSWER:

445





# 优化示例: cryptarithm\_guess

- 优化前: 对于每见过的操作符, 默认使用 absolute difference
- 优化后:
  - 记录操作数的变换
  - 记录已经使用过的操作符
  - 列举常见的候选操作符:  $*$   $+$   $/$   $\#$
  - 排除已经用过的操作符, 按顺序使用第一个候选
  - 操作符 + 操作数变换  $\rightarrow$  答案
  - token长度优化

In Alice's Wonderland, a secret set of  
06-41 = 47  
45)45 = 0  
41)12 = 7  
Now, determine the result for: 43<61

ANSWER:

445

## 优化前

text

```
question operator is not found ... use absolute difference
absolute difference f(43, 61) = |43 - 61| = 18
\boxed{18}
```

所以旧版答案是 18, 错误。

## 优化后

```
Solved example operators: 【)】  $\rightarrow$  absolute difference, 【-】  $\rightarrow$  addition
Used operation names: absolute difference, addition
Common operation candidates ... multiplication, addition, subtraction (a-b), concatenation
We choose the first candidate not already used ... multiplication.
Selected transform: reversed operands and reversed result
multiplication f(34, 16) = ... = 544 -rev $\rightarrow$  445
\boxed{445}
```



- Q&A

